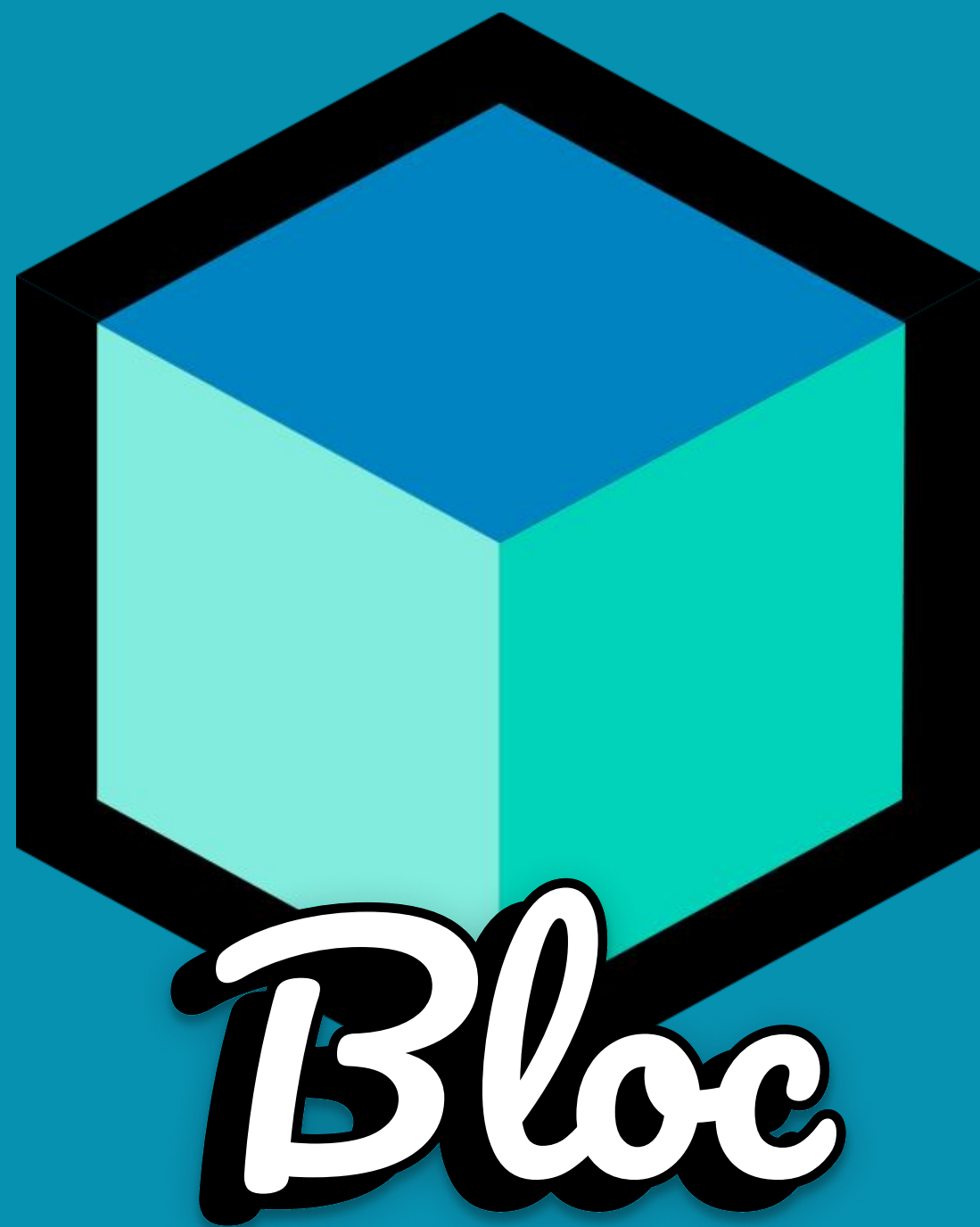
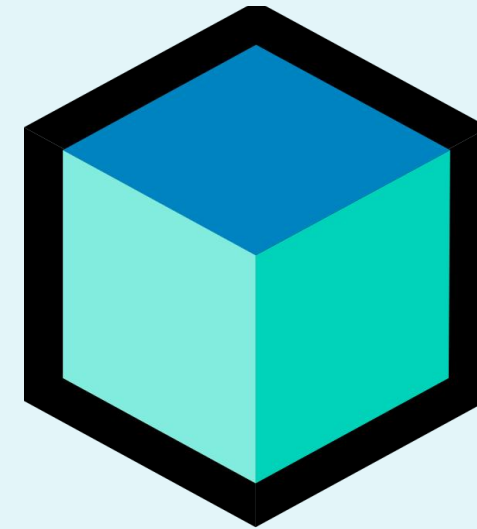




Sobre o Package

O que significa a sigla BLoC?

B: **Business**, Lo: **Logic**, C: **Component**



O que é BLoC?

Ele é um Pattern que tem como objetivo **separar a lógica de negócio da interface do usuário.**

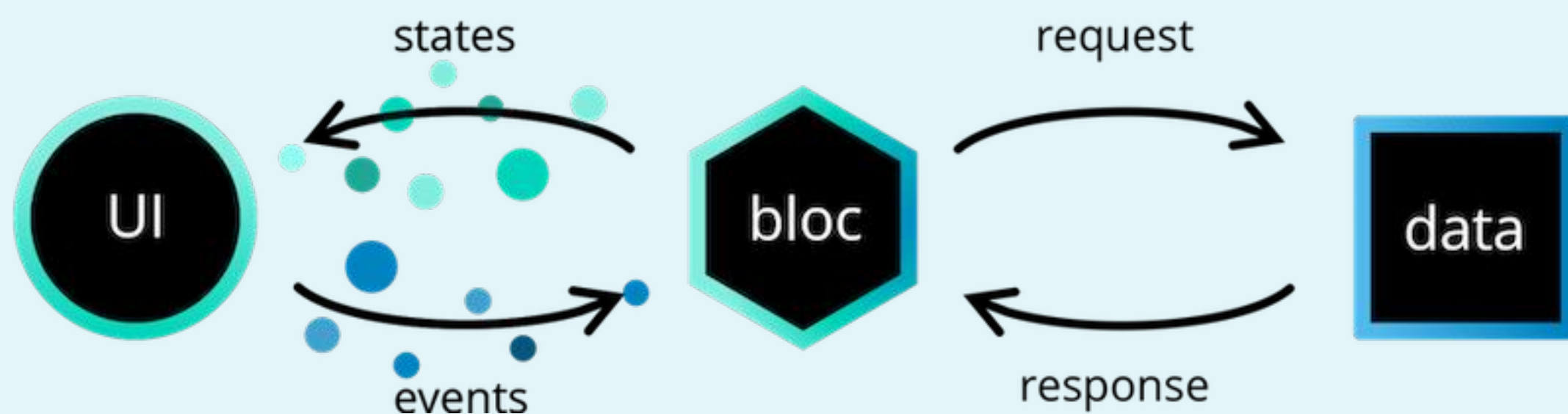
Qual reatividade o package usa?

Stream, sendo uma reatividade de valor único.

BLoC é uma bala de prata?

Não. Para casos de funcionalidades mais simples o ChangeNotifier e ValueNotifier são mais fáceis e são nativos do Flutter.

Como Funciona?



Vamos trabalhar com 3 classes:

Eventos: São **ações do usuário** ou outras alterações que podem afetar o estado do aplicativo.

Estados: Representação da **situação atual do aplicativo**.

Bloc: Onde vai **executar a reatividade**. Recebendo eventos, executa a lógica de negócio e altera o estado dependendo da resposta e notifica quem está escutando.

A entrada e a saída de dados nas streams são feitos em lugares diferentes, como se fosse um canudo.

Polimorfismo

Um fator que faz com que os iniciantes acharem difícil entender o BLoC é não compreender um dos pilares da Orientação a Objetos, o **Polimorfismo**.

Ocorre quando usamos uma **classe base comum** (classe abstrata) para diferentes tipos de estados ou eventos, o BLoC lida com essas classes que herdam dessa classe base **sem precisar saber especificamente qual tipo de evento ou estado está sendo processado**.

Ao herdar da classe abstrata, **a subclasse é tratada como sendo do tipo dessa classe base**. O padrão que vamos utilizar explicado no parágrafo de cima é conhecido como **State Pattern**.

Contextualizando

Vou mostrar na prática o uso do bloc fazendo um pequeno aplicativo de busca de CEP utilizando a famosa API: ViaCep.

Se o CEP existir vai retornar: a rua, estado e a região, e se não existir vai retornar uma mensagem dizendo que não existe

The image displays three mobile application screens for a service named 'VIA CEP'. Each screen features a title bar, an input field for the CEP, a search button, and a results area.

- Screen 1 (Left):** Shows the initial state with the title 'VIA CEP', an input field labeled 'Enter the CEP' with a character count '0/8', a blue 'Search' button, and an empty results box.
- Screen 2 (Middle):** Shows a successful search for the CEP '01001000' (character count '8/8'). The results box contains the following information:
 - CEP: 01001-000
 - ROAD: Praça da Sé
 - STATE: São Paulo
 - REGION: Sudeste
- Screen 3 (Right):** Shows an error state for the CEP '00000000' (character count '8/8'). The results box displays the message 'CEP Not Found'.

Eventos

```
sealed class CepEvents extends Equatable {  
    @override  
    List<Object?> get props => [];  
}
```

```
class FetchCepEvent extends CepEvents {  
    final CepModel cepModel;  
  
    FetchCepEvent(this.cepModel);  
  
    @override  
    List<Object?> get props => [cepModel];  
}
```

Equatable: Package que compara os valores dos atributos da classe, porque se for igual ele vai apontar para a referência já instanciada.

Estados

```
sealed class CepStates extends Equatable {  
    @override  
    List<Object?> get props => [];  
}
```

```
class InitialCepState extends CepStates {}
```

```
class SuccessCepState extends CepStates {  
    final CepModel model;  
  
    SuccessCepState(this.model);  
  
    @override  
    List<Object?> get props => [model];  
}
```

```
class FailureCepState extends CepStates {  
    final String message;  
  
    FailureCepState(this.message);  
  
    @override  
    List<Object?> get props => [message];  
}
```

Bloc

Na classe do Bloc definimos os StreamControllers de entrada e saída, sendo fundamental para a **reatividade** do aplicativo e **permitindo envio de eventos de forma assíncrona para vários ouvintes**.

```
final _inputStream = StreamController<CepEvents>();  
final _outputStream = StreamController<CepStates>();  
  
Sink<CepEvents> get input => _inputStream.sink;  
Stream<CepStates> get stream => _outputStream.stream;
```

Toda vez que a página disparar um evento, esse método que criei: **_blocEventController** vai ser executado.

```
CepBloc() {  
  _inputStream.stream.listen(_blocEventController);  
}
```

Bloc

Nesse método vai **tratar os eventos disparados** da page, **processa a lógica de negócio e dispara os eventos.**

```
void _blocEventController(CepEvents event) async {  
  
  if (event is FetchCepEvent) {  
    try {  
      final endpoint = generateEndpointUtils(event.cepModel.cep);  
      final json = await dio.get(endpoint);  
      final result = CepModel.fromMap(json.data);  
  
      _emit(SuccessCepState(result));  
    } catch (e) {  
      _emit(FailureCepState('CEP Not Found'));  
    }  
  }  
}
```

Método para melhorar a visibilidade

```
void _emit(CepStates state) => _outputStream.add(state);
```

View

Para minha página **ouvir** essa minha classe Bloc precisamos usar o **StreamBuilder**.

```
-StreamBuilder<CepStates>(  
  stream: controller.stream,  
  builder: (context, snapshot) {  
    CepStates? state = snapshot.data;  
  
    return switch (state) {  
      SuccessCepState(:final model) =>  
        CardResultCustom(cepModel: model),  
      FailureCepState(:final message) => Text(  
        message,  
        style: TextStyle.h1,  
      ), // Text  
      InitialCepState() => const SizedBox.shrink(),  
      _ => const SizedBox.shrink()  
    };  
  },  
) // StreamBuilder
```

View

E para finalizar, o disparo de eventos para o Bloc.

```
-ButtonCustom(  
  onTap: () => fetchCep(cepModel),  
) // ButtonCustom
```

```
void fetchCep(CepModel model) {  
  controller.input.add(FetchCepEvent(model));  
}
```

Refatorando StreamController para Flutter_Bloc (package)

Antes

```
class CepBloc {
  CepBloc() {
    _inputStream.stream.listen(_blocEventController);
  }

  final dio = Dio(BaseOptions(baseUrl: apiUrl));

  final _inputStream = StreamController<CepEvents>();
  final _outputStream = StreamController<CepStates>();

  Sink<CepEvents> get input => _inputStream.sink;
  Stream<CepStates> get stream => _outputStream.stream;

  void _blocEventController(CepEvents event) async {
    if (event is FetchCepEvent) {
      try {
        final endpoint = generateEndpointUtils(event.cepModel.cep);
        final json = await dio.get(endpoint);
        final result = CepModel.fromMap(json.data);

        _emit(SuccessCepState(result));
      } catch (e) {
        _emit(FailureCepState('CEP Not Found'));
      }
    }
  }

  void _emit(CepStates state) => _outputStream.add(state);
}
```

Depois

Muito mais simples

```
class CepFlutterBloc extends Bloc<CepEvents, CepStates> {  
  CepFlutterBloc() : super(InitialCepState()) {  
    //  
    final dio = Dio(BaseOptions(baseUrl: apiUrl));  
  
    on<FetchCepEvent>((event, emit) async {  
      try {  
        final endpoint = generateEndpointUtils(event.cepModel.cep);  
        final json = await dio.get(endpoint);  
        final result = CepModel.fromMap(json.data);  
  
        emit(SuccessCepState(result));  
      } catch (e) {  
        emit(FailureCepState('CEP Not Found'));  
      }  
    });  
  }  
}
```

Antes

```
-StreamBuilder<CepStates>(  
  stream: controller.stream,  
  builder: (context, snapshot) {  
    CepStates? state = snapshot.data;  
  
    return switch (state) {  
      SuccessCepState(:final model) =>  
        CardResultCustom(cepModel: model),  
      FailureCepState(:final message) => Text(  
        message,  
        style: TextStyle.h1,  
      ), // Text  
      InitialCepState() => const SizedBox.shrink(),  
      _ => const SizedBox.shrink()  
    };  
  },  
) // StreamBuilder
```

Depois

Mudou quase nada

```
-BlocBuilder<CepFlutterBloc, CepStates>(
  bloc: controller,
  builder: (context, state) {
    return switch (state) {
      SuccessCepState(:final model) =>
        CardResultCustom(cepModel: model),
      FailureCepState(:final message) => Text(
        message,
        style: TextStyle.h1,
      ), // Text
      InitialCepState() => const SizedBox.shrink()
    };
  },
) // BlocBuilder
```

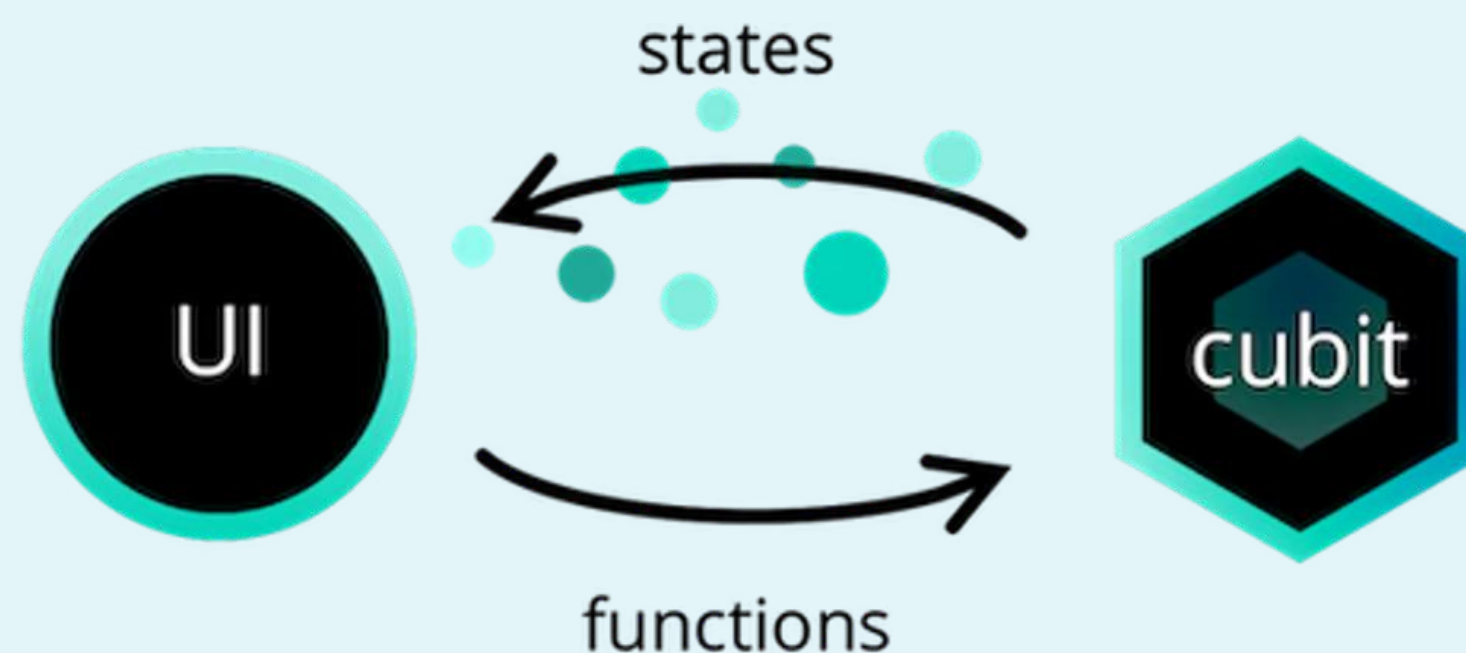


$\mathcal{E}_0$ Cubit?

Cubit

Quase é a mesma coisa que ValueNotifier, só que a diferença é que **um é um Listenable e outro é Stream.**

Qual diferença do Cubit para o Bloc?



- BLoC: Recebe eventos e emite estados.
- Cubit: Emite estados.